CE 222 – *Computer Organization and Assembly Language (COAL)*

Lecture 5

Dr. Asad Mahmood

Jan. 27, 2026

Lecture Outline

- Announcements (Assignment and Quiz 1)
- Outline of Previous Lecture:
  - **Chapter 1 - Computer Abstractions and Technology**
    - 1.6 Performance

- Outline to Today's Lecture:
    - 1.7 The Power Wall
    - 1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors
    - 1.9 Real Stuff: Benchmarking the Intel Core i7
    - 1.10 Going Faster: Matrix Multiply in Python
    - 1.11 Fallacies and Pitfalls
    - 1.12 Concluding Remarks
  - Chapter 2

Chapter 1

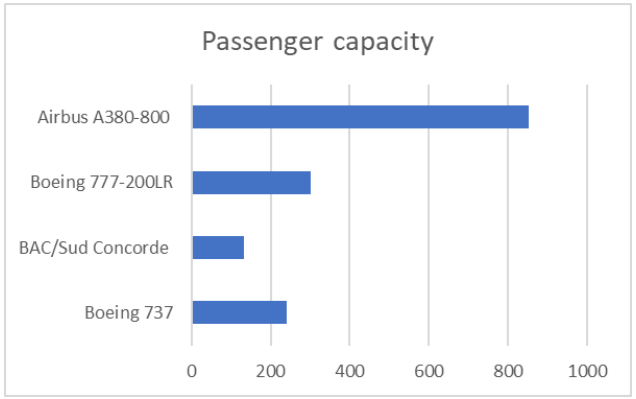
Computer Abstractions and Technology

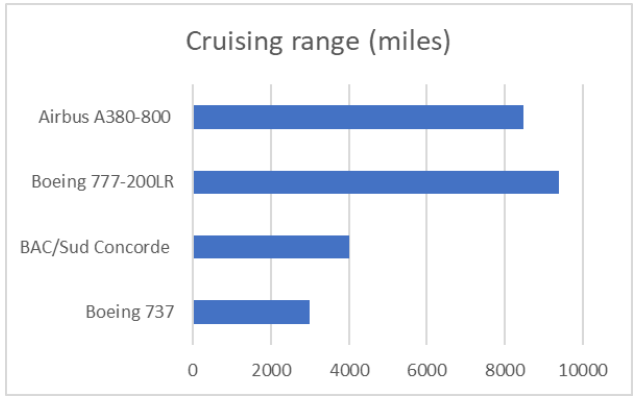
Chapter Outline

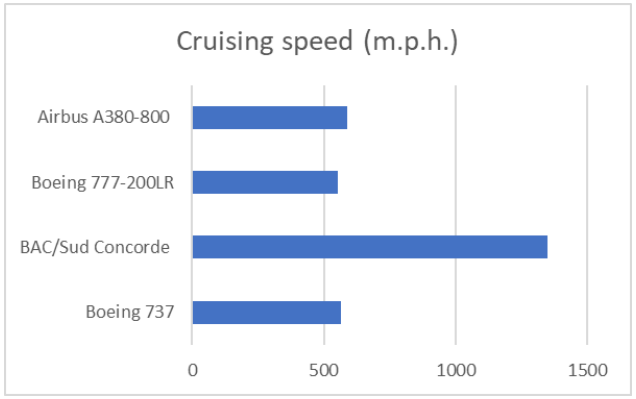
- **1 Computer Abstractions and Technology**
  - 1.1 Introduction
  - 1.2 Seven Great Ideas in Computer Architecture
  - 1.3 Below Your Program
  - 1.4 Under the Covers
  - 1.5 Technologies for Building Processors and Memory
  - 1.6 Performance
  - 1.7 The Power Wall
  - 1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors
  - 1.9 Real Stuff: Benchmarking the Intel Core i7
  - 1.10 Going Faster: Matrix Multiply in Python
  - 1.11 Fallacies and Pitfalls
  - 1.12 Concluding Remarks
  - 1.13 Historical Perspective and Further Reading

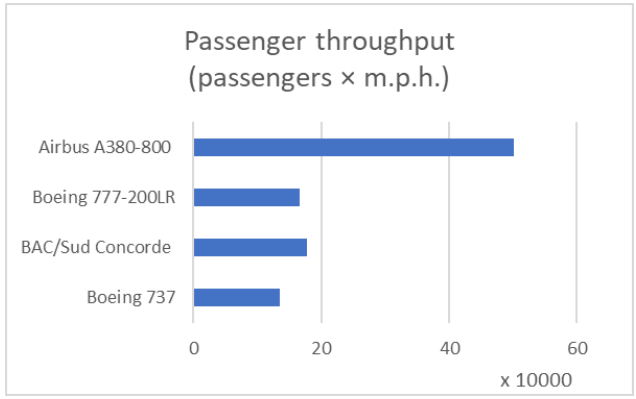
Defining Performance

- Very important to define computer performance but not easy!
- Which airplane has the best performance?









Response Time and Throughput

- We can define computer performance in several distinct ways
- Response time
  - How long it takes to do a task
  - Focus of an individual user
- Throughput
  - Total work done per unit time e.g., Total tasks / transactions/… per hour
  - Focus of a data-center/server
- How are response time and throughput affected by
  1. Case 1 - Replacing the processor with a faster version?
  2. Case 2 - Adding more processors?
- We'll focus on response time for now…

Relative Performance

- Let us define the *Performance* of a computer in terms of its Execution Time
- What it means when one says “X is *n* time faster than Y”?
- Example: If computer A runs a program in 10 seconds and computer B runs the same program in 15 seconds, how much faster is A than B?

Measuring Execution Time

- Execution time (Seconds/Program) can be measured in a number of ways, e.g.
  1. Elapsed time = Total response time, including all aspects i.e. Processing, I/O, OS overhead, idle time

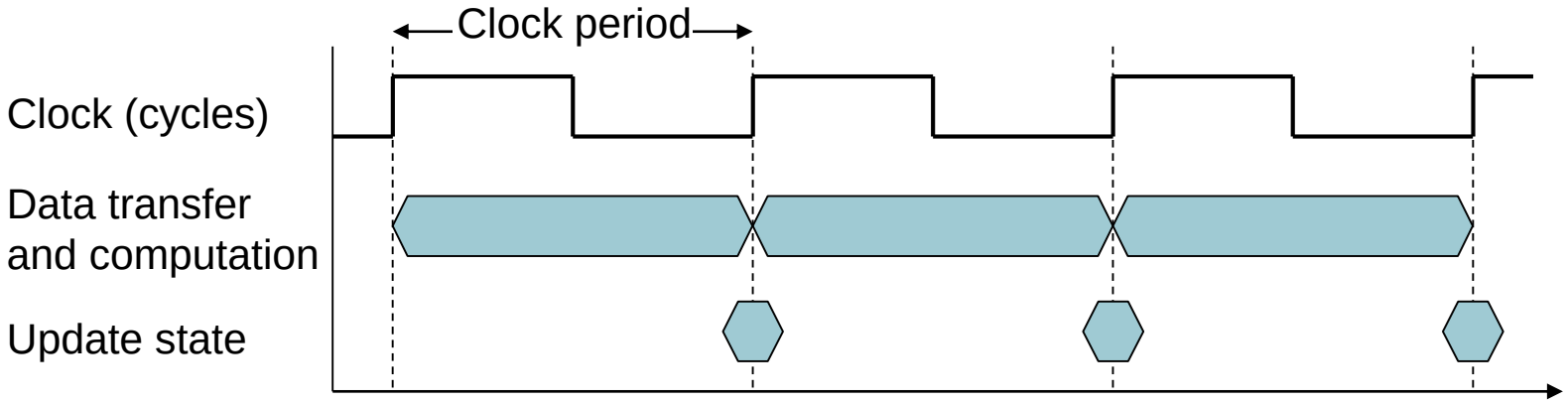
  2. CPU time =Time spent processing a given job without including I/O time, other jobs' shares
     - Comprises user CPU time and system CPU time

- Which one of the above two is experienced by the user?

- We will focus on CPU performance in this chapter

CPU Clocking

- Operation of digital hardware governed by a constant-rate clock, which can be used to define CPU time



- Clock period (s) and clock frequency / rate (cycles/s)
- Relating Clock period/Freq. with CPU time
- How performance can be improved?
- Often a tradeoff involved!

CPU Time Example

- Computer A: 2GHz clock, 10s CPU time
- Designing Computer B
  - Aim for 6s CPU time
  - Can do faster clock, but causes 1.2 × clock cycles
- How fast must Computer B clock be?

Instruction Count and CPI

- Instruction Count (# of Instructions) for a program
  - Determined by program, ISA and compiler
- Relating instruction count with the CPU execution time
- Average cycles per instruction (CPI) affected by instruction mix

Components of performance	Units of measure
CPU execution time for a program	Seconds for the program
Instruction count	Instructions executed for the program
Clock cycles per instruction (CPI)	Average number of clock cycles per instruction
Clock cycle time	Seconds per clock cycle

FIGURE 1.15 The basic components of performance and how each is measured.

CPI Example

- Computer A: Cycle Time = 250ps, CPI = 2.0
- Computer B: Cycle Time = 500ps, CPI = 1.2
- Same ISA
- Which is faster, and by how much?

- CPI in more detail

CPI Example

- For a high-level language statement, a compiler writer is considering two code sequences for a computer that require the following instruction counts for 3 classes of instructions A, B and C

Class	A	B	C
CPI for class	1	2	3
IC in sequence 1	2	1	2
IC in sequence 2	4	1	1

- Which code sequence executes the most instructions?
- What is the CPI for each sequence?
- Which will be faster?

Performance Summary

The BIG Picture

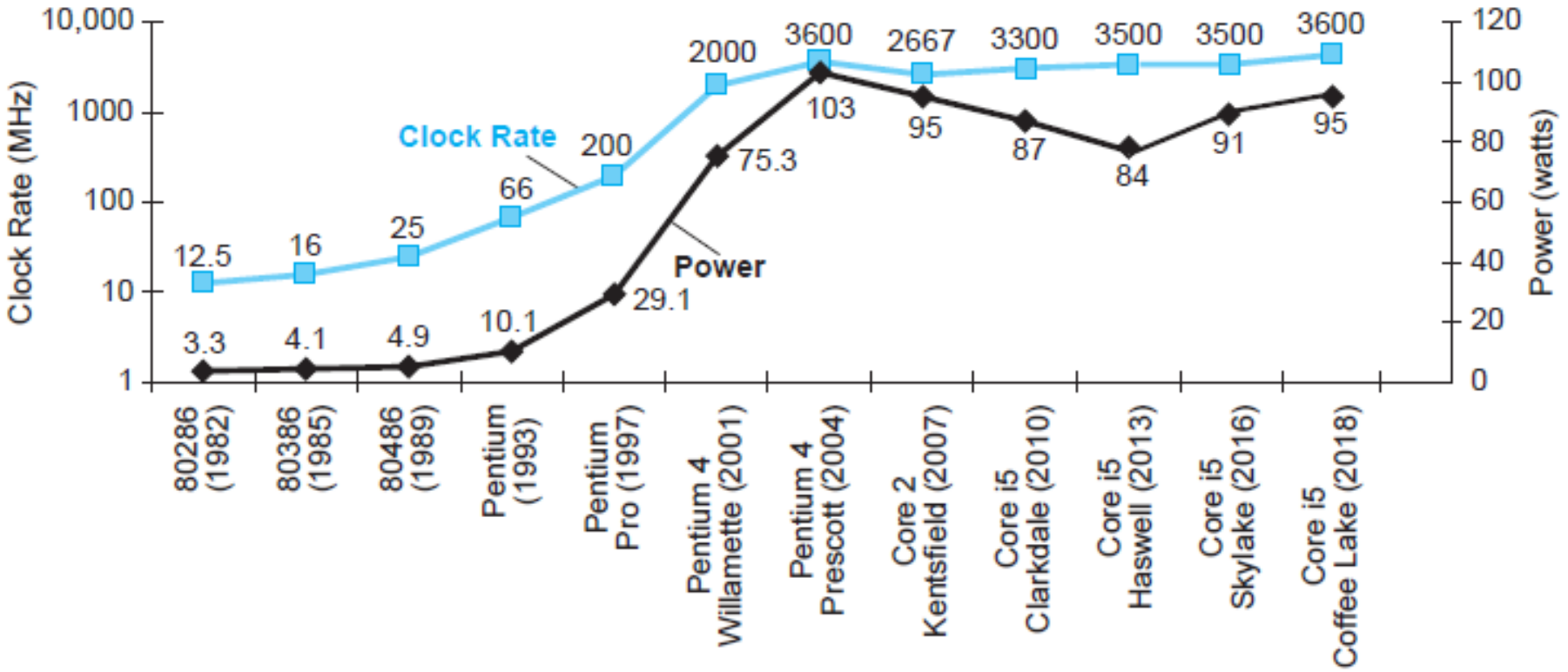
$$\text{CPU Time} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}}$$

- Performance depends on
  - Algorithm: affects IC, possibly CPI
  - Programming language: affects IC, CPI
  - Compiler: affects IC, CPI
  - Instruction set architecture: affects IC, CPI, T_c

Chapter Outline

- **1 Computer Abstractions and Technology**
  - 1.1 Introduction
  - 1.2 Seven Great Ideas in Computer Architecture
  - 1.3 Below Your Program
  - 1.4 Under the Covers
  - 1.5 Technologies for Building Processors and Memory
  - 1.6 Performance
  - 1.7 The Power Wall
  - 1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors
  - 1.9 Real Stuff: Benchmarking the Intel Core i7
  - 1.10 Going Faster: Matrix Multiply in Python
  - 1.11 Fallacies and Pitfalls
  - 1.12 Concluding Remarks
  - 1.13 Historical Perspective and Further Reading

Powe



- In CMOS IC technology

$$\text{Power} = \text{Capacitive load} \times \text{Voltage}^2 \times \text{Frequency}$$

×30 | 5V → 1V | ×1000

Reducing Power

- Suppose a new CPU has
  - 85% of capacitive load of old CPU
  - 15% voltage and 15% frequency reduction

$$\frac{P_{new}}{P_{old}} = \frac{C_{old} \times 0.85 \times (V_{old} \times 0.85)^2 \times F_{old} \times 0.85}{C_{old} \times V_{old}^{\ 2} \times F_{old}} = 0.85^4 = 0.52$$

- The power wall
  - We can't reduce voltage further
  - We can't remove more heat
- How else can we improve performance?

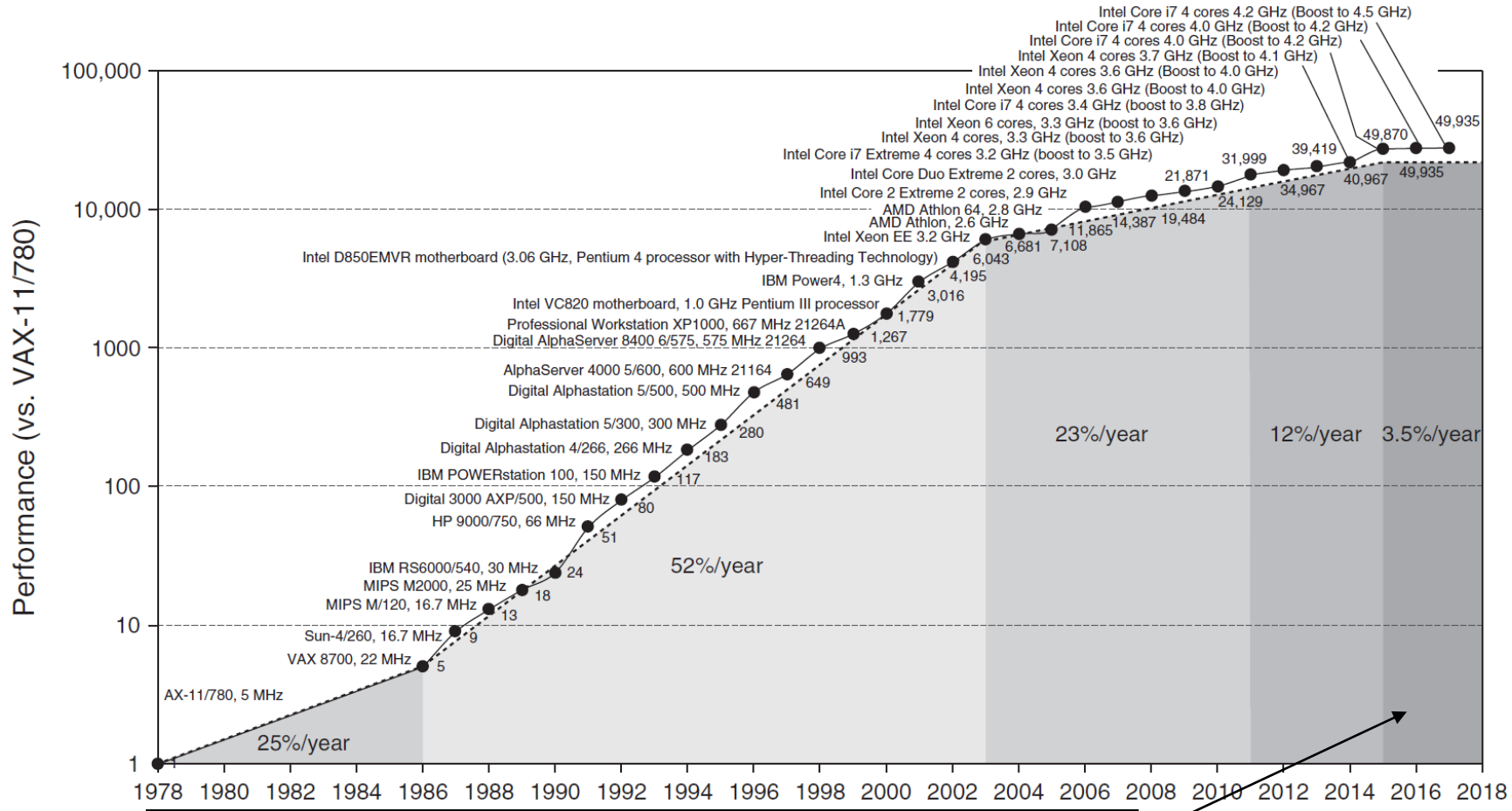
Chapter Outline

- **1 Computer Abstractions and Technology**
  - 1.1 Introduction
  - 1.2 Seven Great Ideas in Computer Architecture
  - 1.3 Below Your Program
  - 1.4 Under the Covers
  - 1.5 Technologies for Building Processors and Memory
  - 1.6 Performance
  - 1.7 The Power Wall
  - 1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors
  - 1.9 Real Stuff: Benchmarking the Intel Core i7
  - 1.10 Going Faster: Matrix Multiply in Python
  - 1.11 Fallacies and Pitfalls
  - 1.12 Concluding Remarks
  - 1.13 Historical Perspective and Further Reading

1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors

> *“ Up to now, most software has been like music written for a solo performer; with the current generation of chips we’re getting a little experience with duets and quartets and other small ensembles; but scoring a work for large orchestra and chorus is a different kind of challenge.”*
>
> Brian Hayes, *Computing in a Parallel Universe*, 2007.

Uniprocessor Performance



Constrained by power, instruction-level parallelism, memory latency

Multiprocessors

- Multicore microprocessors
  - More than one processor per chip
- Requires explicitly parallel programming
  - Compare with instruction level parallelism
    - Hardware executes multiple instructions at once
    - Hidden from the programmer
  - Programming for multicore -> Hard to do
    - Programming for performance/speed
    - Load balancing
    - Optimizing communication and synchronization

Chapter Outline

- **1 Computer Abstractions and Technology**
  - 1.1 Introduction
  - 1.2 Seven Great Ideas in Computer Architecture
  - 1.3 Below Your Program
  - 1.4 Under the Covers
  - 1.5 Technologies for Building Processors and Memory
  - 1.6 Performance
  - 1.7 The Power Wall
  - 1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors
  - 1.9 Real Stuff: Benchmarking the Intel Core i7
  - 1.10 Going Faster: Matrix Multiply in Python
  - 1.11 Fallacies and Pitfalls
  - 1.12 Concluding Remarks
  - 1.13 Historical Perspective and Further Reading

SPEC CPU Benchmark

- Programs used to measure performance
  - Supposedly typical of actual workload
- Standard Performance Evaluation Corp (SPEC)
  - Develops benchmarks for CPU, I/O, Web, …
- SPEC CPU2006
  - Elapsed time to execute a selection of programs
    - Negligible I/O, so focuses on CPU performance
  - Normalize relative to reference machine
  - Summarize as geometric mean of performance ratios
    - CINT2006 (integer) and CFP2006 (floating-point)

$$\sqrt[n]{\prod_{i=1}^{n} \text{Execution time ratio}_i}$$

SPECspeed 2017 Integer benchmarks on a 1.8 GHz Intel Xeon E5-2650L

Description	*Name*	*Instruction Count x 10^9*	*CPI*	*Clock cycle time (seconds x 10^–9)*	*Execution Time (seconds)*	*Reference Time (seconds)*	*SPECratio*
Perl interpreter	perlbench	2684	0.42	0.556	627	1774	2.83
GNU C compiler	gcc	2322	0.67	0.556	863	3976	4.61
Route planning	mcf	1786	1.22	0.556	1215	4721	3.89
Discrete Event simulation - computer network	omnetpp	1107	0.82	0.556	507	1630	3.21
XML to HTML conversion via XSLT	xalancbmk	1314	0.75	0.556	549	1417	2.58
Video compression	x264	4488	0.32	0.556	813	1763	2.17
Artificial Intelligence: alpha-beta tree search (Chess)	deepsjeng	2216	0.57	0.556	698	1432	2.05
Artificial Intelligence: Monte Carlo tree search (Go)	leela	2236	0.79	0.556	987	1703	1.73
Artificial Intelligence: recursive solution generator (Sudoku)	exchange2	6683	0.46	0.556	1718	2939	1.71
General data compression	xz	8533	1.32	0.556	6290	6182	0.98
Geometric mean							2.36

SPEC Power Benchmark

- Given the increasing importance of energy and power, SPEC added a benchmark to measure power. at different workload levels
- SPECpower_ssj2008 for Xeon E5-2650L

Target Load %	Performance (ssj_ops)	Average Power (watts)
100%	4,864,136	347
90%	4,389,196	312
80%	3,905,724	278
70%	3,418,737	241
60%	2,925,811	212
50%	2,439,017	183
40%	1,951,394	160
30%	1,461,411	141
20%	974,045	128
10%	485,973	115
0%	0	48
Overall Sum	26,815,444	2,165
∑ssj_ops / ∑power =		12,385

Chapter Outline

- **1 Computer Abstractions and Technology**
  - 1.1 Introduction
  - 1.2 Seven Great Ideas in Computer Architecture
  - 1.3 Below Your Program
  - 1.4 Under the Covers
  - 1.5 Technologies for Building Processors and Memory
  - 1.6 Performance
  - 1.7 The Power Wall
  - 1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors
  - 1.9 Real Stuff: Benchmarking the Intel Core i7
  - 1.10 Going Faster: Matrix Multiply in Python
  - 1.11 Fallacies and Pitfalls
  - 1.12 Concluding Remarks
  - 1.13 Historical Perspective and Further Reading

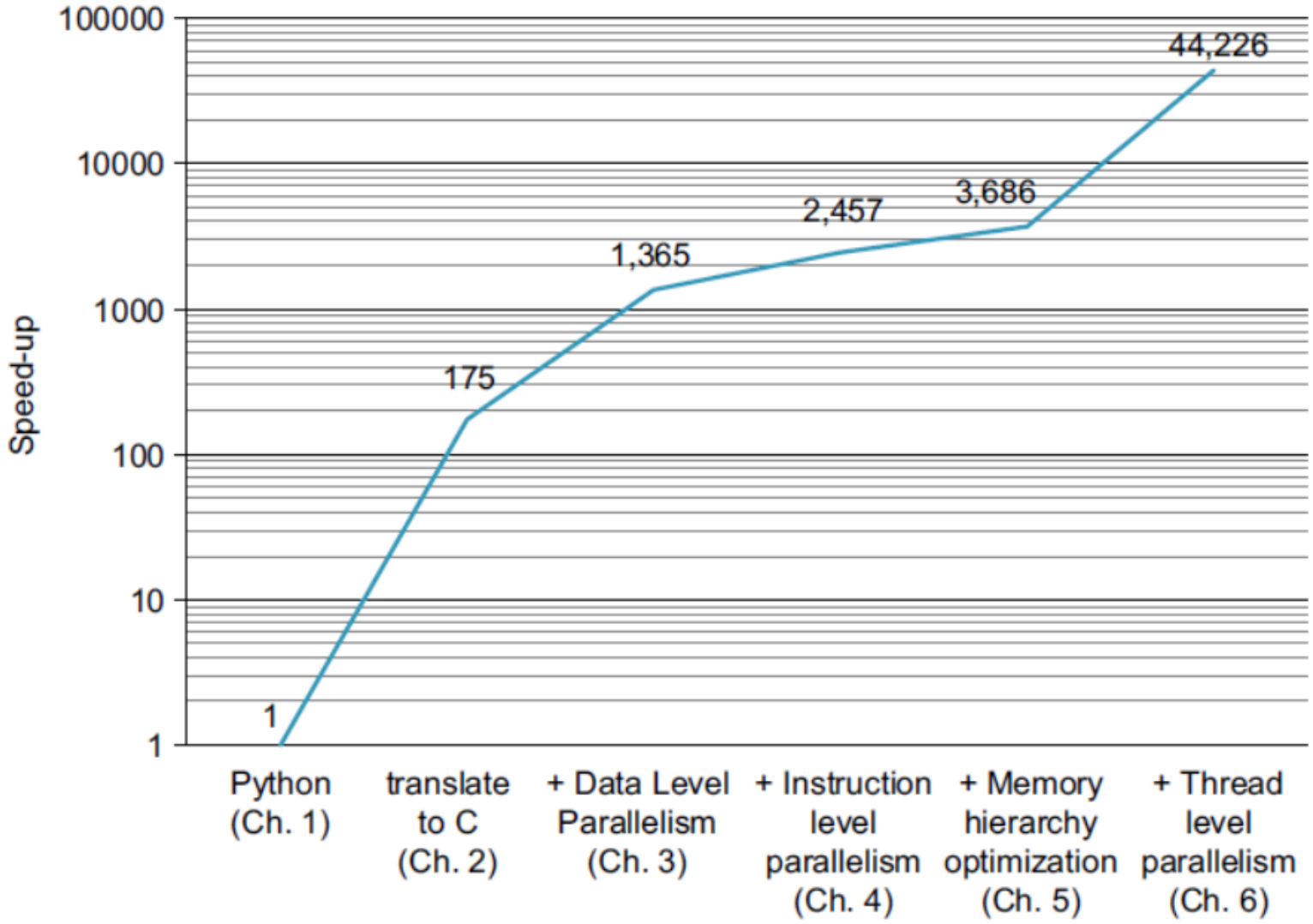
Going Faster: Matrix Multiply in Python

- To show the impact of different strategies in this book to improve performance, a matrix multiplication example is used with same Hardware (n1-standard-96 server in Google Cloud Engine, which has two Intel Skylake Xeon chips, and each chip has 24 processors or cores)

```
for i in xrange(n):
    for j in xrange(n):
        for k in xrange(n):
            C[i][j] += A[i][k] * B[k][j]
```

- When Python code is used
  - For matrix size 960*960 -> 5 minutes
  - For matrix size 4096*4096 -> 6 hours!

Going Faster: Matrix Multiply in Python



Chapter Outline

- **1 Computer Abstractions and Technology**
  - 1.1 Introduction
  - 1.2 Seven Great Ideas in Computer Architecture
  - 1.3 Below Your Program
  - 1.4 Under the Covers
  - 1.5 Technologies for Building Processors and Memory
  - 1.6 Performance
  - 1.7 The Power Wall
  - 1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors
  - 1.9 Real Stuff: Benchmarking the Intel Core i7
  - 1.10 Going Faster: Matrix Multiply in Python
  - 1.11 Fallacies and Pitfalls
  - 1.12 Concluding Remarks
  - 1.13 Historical Perspective and Further Reading

Fallacies and Pitfalls

- *"Science must begin with myths, and the criticism of myths"* - Sir Karl Popper, *The Philosophy of Science*,1957

- Each chapter of this book has some fallacies and pitfalls mentioned at the end to clear misconceptions.
  - Fallacy: commonly held mis-concpetions
  - Pitfall: Mistake e.g. generalization of a concept which is only true in a limited context

Pitfall

- *Pitfall: Improving an aspect of a computer and expecting a proportional improvement in overall performance*

Amdahl's Law

$$T_{improved} = \frac{T_{affected}}{improvement\ factor} + T_{unaffected}$$

- Example: If a programs takes 100s to run, how much overall improvement in execution time for a program do we get if we reduce the multiplication time by 50%. Consider the cases of:
  - Case 1: Multiplications take 80s out of 100s
  - Case 2: Multiplications take 20s out of 100s

Fallacy

- *Fallacy: Computers at low utilization use little power.*
- Look back at i7 power benchmark
  - At 100% load: 258W
  - At 50% load: 170W (66%)
  - At 10% load: 121W (47%)
- Google data center
  - Mostly operates at 10% – 50% load
  - At 100% load less than 1% of the time
- Consider designing processors to make power proportional to load

Pitfall

- *Pitfall: Using a subset of the performance equation as a performance metric.*
- MIPS: Millions of Instructions Per Second
  - Doesn’t account for
    - Differences in ISAs between computers
    - Differences in complexity between instructions
- Example:
  - Which computer has higher MIPS?
  - Which computer is faster?

Measurement	Computer A	Computer B
Instruction count	10 billion	8 billion
Clock rate	4 GHz	4 GHz
CPI	1.0	1.1

Chapter Outline

- **1 Computer Abstractions and Technology**
  - 1.1 Introduction
  - 1.2 Seven Great Ideas in Computer Architecture
  - 1.3 Below Your Program
  - 1.4 Under the Covers
  - 1.5 Technologies for Building Processors and Memory
  - 1.6 Performance
  - 1.7 The Power Wall
  - 1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors
  - 1.9 Real Stuff: Benchmarking the Intel Core i7
  - 1.10 Going Faster: Matrix Multiply in Python
  - 1.11 Fallacies and Pitfalls
  - 1.12 Concluding Remarks
  - 1.13 Historical Perspective and Further Reading

Concluding Remarks

- Cost/performance is improving
  - Due to underlying technology development
- Hierarchical layers of abstraction
  - In both hardware and software
- Instruction set architecture
  - The hardware/software interface
- Execution time: the best performance measure
- Power is a limiting factor
  - Use parallelism to improve performance

Chapter Outline

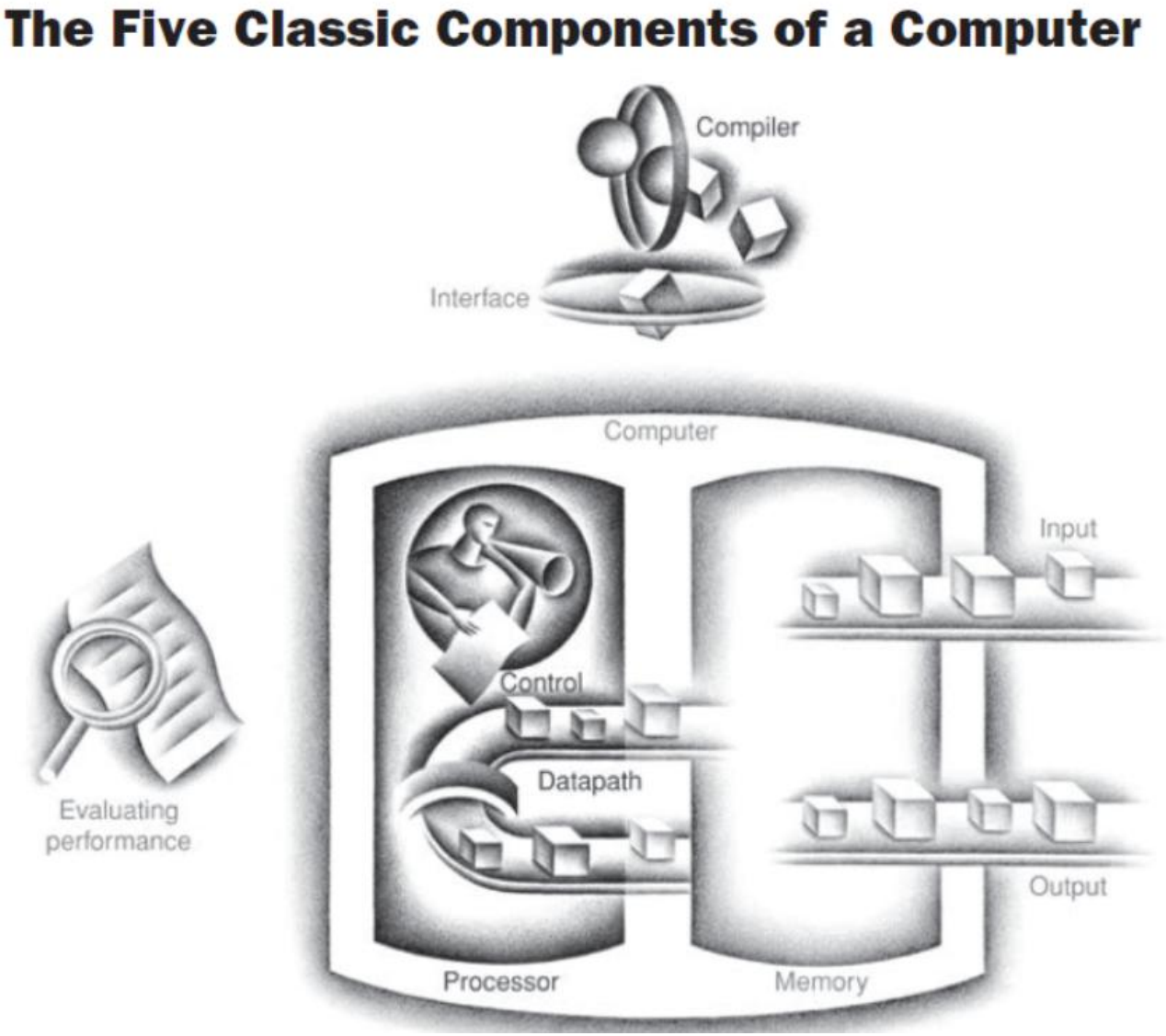
- **1 Computer Abstractions and Technology**
  - 1.1 Introduction
  - 1.2 Seven Great Ideas in Computer Architecture
  - 1.3 Below Your Program
  - 1.4 Under the Covers
  - 1.5 Technologies for Building Processors and Memory
  - 1.6 Performance
  - 1.7 The Power Wall
  - 1.8 The Sea Change: The Switch from Uniprocessors to Multiprocessors
  - 1.9 Real Stuff: Benchmarking the Intel Core i7
  - 1.10 Going Faster: Matrix Multiply in Python
  - 1.11 Fallacies and Pitfalls
  - 1.12 Concluding Remarks
  - 1.13 Historical Perspective and Further Reading –Self-Study

The End

Chapter 2

Instructions: Language of the Computer

Chapter Theme



Chapter Outline

- **2 Instructions: Language of the Computer**
  - 2.1 Introduction
  - 2.2 Operations of the Computer Hardware
  - 2.3 Operands of the Computer Hardware
  - 2.4 Signed and Unsigned Numbers
  - 2.5 Representing Instructions in the Computer
  - 2.6 Logical Operations
  - 2.7 Instructions for Making Decisions
  - 2.8 Supporting Procedures in Computer Hardware
  - 2.9 Communicating with People
  - 2.10 RISC-V Addressing for Wide Immediates and Addresses
  - 2.11 Parallelism and Instructions: Synchronization
  - 2.12 Translating and Starting a Program
  - 2.13 A C Sort Example to Put it All Together
  - 2.14 to 2.24 …..

Introduction

- To command a computer's hardware, you must speak its language
  - The language of computer is a set of instructions called as the instruction set.
- Different computers have different instruction sets
  - But with many aspects in common
  - Examples: RISC-V, MIPS, x86 etc.
- Many computers have a very simple instruction sets
  - Simplified implementation
- Instruction Set Architecture (ISA) = Instruction Set + rules +data types, addressing modes etc.
- ISA is a consequence of the Stored-Program concept.

Chapter Outline

- **2 Instructions: Language of the Computer**
  - 2.1 Introduction
  - 2.2 Operations of the Computer Hardware
  - 2.3 Operands of the Computer Hardware
  - 2.4 Signed and Unsigned Numbers
  - 2.5 Representing Instructions in the Computer
  - 2.6 Logical Operations
  - 2.7 Instructions for Making Decisions
  - 2.8 Supporting Procedures in Computer Hardware
  - 2.9 Communicating with People
  - 2.10 RISC-V Addressing for Wide Immediates and Addresses
  - 2.11 Parallelism and Instructions: Synchronization
  - 2.12 Translating and Starting a Program
  - 2.13 A C Sort Example to Put it All Together
  - 2.14 to 2.24 …..